

Contents

guide_01_ai_agents_that_matter	1
0. 这篇论文一句话	2
1. 当时的历史语境	2
2. 论文结构地图	2
3. 先建立正确问题意识	3
4. 论文如何定义 agent	4
5. 五个核心贡献	4
5.1 贡献一: agent evaluation 必须 cost-controlled	4
5.2 贡献二: accuracy 和 cost 应该联合优化	5
5.3 贡献三: model evaluation 和 downstream evaluation 不同	5
5.4 贡献四: agent benchmark 很容易产生 shortcut	6
5.5 贡献五: agent evaluation 缺乏标准化和可复现性	7
6. Figure 1: HumanEval 上的成本受控比较	7
7. Pareto frontier 怎么读	8
8. fixed cost 和 variable cost	9
9. Figure 2: HotPotQA 和 DSPy 联合优化	10
10. NovelQA: benchmark task shape 会误导成本	11
11. Table 1: holdout 必须匹配泛化目标	12
12. WebArena 和 STeP case study	12
13. human-in-the-loop evaluation	13
14. 可复现性为什么在 agent 中更难	14
15. 全文机制图	14
16. 数据形状和评测对象	15
17. 数学化理解	17
17.1 成本受控效用	17
17.2 Pareto 支配	17
17.3 使用量摊销	18
17.4 downstream cost 不是 proxy	18
18. 本地代码例子	18
19. 和本地 agent-graduation 代码的关系	19
20. 30 到 60 分钟本地实验	20
21. 证据链逐段复盘	20
22. 论文没有证明什么	21
23. 对现在学习 LLM agent 的意义	21
24. 用 AI agent 正确学习这篇	22
25. 闭卷掌握检查	23
26. 最小复述模板	23

guide_01_ai_agents_that_matter

原论文: AI Agents That Matter

本地原文 PDF: [learning/agent-graduation/paper/01_ai_agents_that_matter.pdf](#)

作者: Sayash Kapoor, Benedikt Stroebl, Zachary S. Siegel, Nitya Nadgir, Arvind Narayanan

机构: Princeton University

日期: 2024-07-02

类型: agent evaluation, benchmark methodology, cost-controlled evaluation

0. 这篇论文一句话

这篇论文的核心不是提出一个新 agent 架构，而是给 agent 领域泼一盆非常有用的冷水: 如果评测不控制成本、不使用强 baseline、不设计正确 holdout、不标准化复现流程，那么很多看起来 state-of-the-art 的 agent 进步，可能只是更贵的重试、更会利用 benchmark 漏洞，或者更依赖不可复现的评测细节。

它的标题里说“agents that matter”，意思不是“所有 agent 都不重要”，而是要把真正有现实价值的 agent 和 leaderboard 上的漂亮分数区分开。

你读完这篇应该形成一个新的工程直觉:

- 看到 agent 论文先问成本，不先问分数。
- 看到复杂规划和反思模块先问强 baseline，不先相信故事。
- 看到 benchmark 排行榜先问 holdout 层级，不先相信泛化。
- 看到 agent demo 先问可复现协议，不先被界面打动。
- 看到下游产品选型先用 dollar cost 和 token count，不只用参数量或模型名。

1. 当时的历史语境

2023 到 2024 年，LLM agent 变成非常热的方向。ReAct 把推理和行动交织起来，Toolformer 让模型学习调用工具，WebArena、SWE-bench、GAIA、AgentBench 等 benchmark 让 agent 有了排行榜。很多系统开始加入 planning、reflection、memory、tool use、multi-agent、browser control、code execution 等模块。

这个阶段最容易出现两个错觉。

第一，复杂就是智能。一个 agent 如果有 planner、critic、executor、memory、tool router、self-reflection，看起来就比一个普通模型强很多。但如果它只是多调用几次 GPT-4，多花了几十倍成本，它的 gain 未必来自架构。

第二，benchmark 分数就是现实价值。一个 agent 在 WebArena 或 HumanEval 上得分更高，不一定意味着它在真实业务中更可靠。它可能只是在固定测试集上学会了捷径，或者评测脚本刚好偏向某种输出格式。

这篇论文出现在这个背景下，作用像一次 agent evaluation 的地基检查。作者不是反对 agent，而是要求 agent 领域把进步讲清楚:

- 进步到底来自架构，还是来自更多采样和更贵模型。
- 进步是否在成本受控时仍成立。
- benchmark 是否能代表真实部署场景。
- agent 是否能在任务、网站、环境变化之后仍有效。
- 论文代码和数据是否足以复现结果。

2. 论文结构地图

原文主线非常清楚，可以按六个部分读。

第 1 节 Introduction:

- 给出五个贡献。
- 解释为什么 agent evaluation 和普通 LLM evaluation 不一样。

- 强调论文目标是推动现实中有用的 agent，而不只是 benchmark 上准确的 agent。

第 2 节 Cost-controlled agent evaluations:

- 主张 agent 评测必须控制成本。
- 在 HumanEval 上比较复杂 agent 和简单 baseline。
- 提出 retry、warming、escalation 三个简单 baseline。
- 用 Figure 1 展示 accuracy-cost Pareto 曲线。

第 3 节 Jointly optimizing cost and accuracy:

- 把 agent 设计变成同时优化准确率和成本的问题。
- 区分 fixed cost 和 variable cost。
- 修改 DSPy，在 HotPotQA 上做 accuracy-cost 联合优化。
- 用 Figure 2 说明 joint optimization 可以保持准确率同时降低成本。

第 4 节 Model evaluation vs downstream evaluation:

- 区分模型开发者和下游开发者的评测需求。
- 模型评测可以使用 compute 或参数量等 proxy。
- 下游评测应该关注真实 dollar cost 和 token count。
- 用 NovelQA 说明模型 benchmark 可能误导下游应用。

第 5 节 Benchmark shortcuts:

- 指出 agent benchmark 容易被 shortcut 和 overfit。
- 提出四个 generality level。
- 用 Table 1 说明不同泛化目标需要不同 holdout。
- 用 WebArena 和 STeP 作为 case study。
- 讨论 human-in-the-loop evaluation 的缺失。

第 6 节 Standardization and reproducibility:

- 总结 agent evaluation 不可复现的五类根因。
- 讨论 HumanEval 和 WebArena 的复现问题。
- 解释为什么 HELM 和 LM Eval Harness 这类 model evaluation 工具还不够。
- 呼吁建立 agent-specific evaluation standard。

附录:

- 包含成本模型、HumanEval 细节、agent benchmark survey、NovelQA 成本分析、WebArena 和 HumanEval 复现细节。
- Table A4 汇总 17 个 agent benchmark 的 holdout 情况。
- Table A5 支持 NovelQA 成本分析。
- Table A6 展示标准化不足导致的具体复现问题。
- Table A7 给出 HumanEval 复现实验的数值范围。

3. 先建立正确问题意识

这篇论文真正关心的问题是:

我们怎样判断一个 agent 真的更有用?

不是:

它的准确率是否最高?

而是：

- 在同等成本下它是否更准确？
- 在同等准确率下它是否更便宜？
- 它是否击败了足够强的简单 baseline？
- 它是否在正确 holdout 上泛化？
- 它是否可以被别人复现？
- 它是否匹配真实用户工作流？

如果一个 agent 只在某个 benchmark 上高分，但成本高、baseline 弱、holdout 错、复现难、真实使用方式不同，那么这个 agent “matters” 的证据就很弱。

注意，作者并没有说复杂 agent 架构一定无用。他们说的是：当前很多证据不足以说明复杂架构本身带来了进步。这个区分很重要。

4. 论文如何定义 agent

论文没有采用一个二元定义。它强调 agentic 是一个 spectrum，而不是“是 agent”或“不是 agent”的硬分类。

可以从三组维度理解 agentic 程度。

第一组是环境和目标：

- 环境越复杂，agentic 程度越高。
- 任务 horizon 越长，agentic 程度越高。
- 目标越开放、越自然语言化，agentic 程度越高。
- 需要跨多个步骤和状态恢复的任务，比一次问答更 agentic。

第二组是用户界面和监督：

- 人类监督越少，agentic 程度越高。
- agent 自主选择 action 的空间越大，风险也越大。
- 如果每一步都由用户确认，它更像辅助工具。
- 如果 agent 可以长时间自主操作环境，它更像自动化代理。

第三组是系统设计：

- 是否使用工具。
- 是否 planning。
- 是否 memory。
- 是否动态控制流。
- 是否能根据 observation 改变下一步行为。
- 是否能执行真实世界或模拟世界里的 action。

这个定义对新手很有帮助，因为它避免了一个常见误区：不是套了 ReAct prompt 就自动成为有价值 agent，也不是加了 planner 就自动代表智能提升。agent 的价值取决于任务、环境、成本和评测协议。

5. 五个核心贡献

5.1 贡献一：agent evaluation 必须 cost-controlled

很多 agent 论文只报告 accuracy，不报告 cost。作者认为这会制造严重误导，因为 agent 往往通过多次调用 LLM、多轮工具执行、多次反思和调试来提高准确率。

如果只看准确率，那么一个方法可以通过无限 retry 变得更好。这个现象在代码生成尤其明显。论文提到 AlphaCode 这类结果说明，采样次数从少量变成大量后，准确率可以显著上升。但如果你把一百万次采样也算作同一个“方法”，那 accuracy 本身就不再能说明架构创新。

作者在 HumanEval 上重新评估多个 coding agent，并加入三个简单 baseline:

- retry: 如果 visible tests 失败，就用同一个模型再次生成，最多五次。
- warming: 也是 retry，但温度从 0 逐步升到 0.5，提高随机性。
- escalation: 先用便宜模型，失败时升级到更贵更强的模型。

这些 baseline 没有复杂 planning、reflection 或 tree search，却在 HumanEval 上和复杂 agent 非常有竞争力，甚至在 Pareto 意义上更好。

这带来一个强烈结论:

如果复杂 agent 没有和 retry、warming、escalation 比较，就不能证明复杂 agent 架构本身造成了提升。

5.2 贡献二: accuracy 和 cost 应该联合优化

作者不是只说“报告成本”，而是进一步说: 成本应该进入设计目标。

一个 agent 设计点可以表示成:

agent_config -> accuracy, variable_cost, fixed_cost, latency

如果一个 agent 更准确也更便宜，那它显然更好。如果一个 agent 更准确但更贵，就要看是否值得。如果一个 agent 准确率相同但更贵，通常没有理由选它。

这就是 Pareto frontier 的意义。Pareto frontier 上的点没有被其他点同时在准确率和成本上支配。一个被支配的点，不管故事多漂亮，都不是好设计。

作者在 HotPotQA 上修改 DSPy 的优化过程，让优化器不只追求准确率，还搜索更便宜的配置。搜索空间包括:

- 每个 module 的 temperature。
- few-shot examples 的数量。
- 具体选择哪些 examples。
- 是否加入 formatting instructions。

实验结果显示, joint optimization 可以保持接近准确率, 同时显著降低 variable cost。论文报告 GPT-3.5 配置可降低 53% variable cost, Llama-3-70B 配置可降低 41% cost。由于 joint optimization 有 fixed cost, 作者还讨论了使用量足够大以后什么时候 amortize。

5.3 贡献三: model evaluation 和 downstream evaluation 不同

模型开发者和下游开发者问的是不同问题。

模型开发者想知道:

- 我的架构是不是更好。
- 我的数据配方是不是更好。
- 我的训练 compute 是否被更有效使用。
- 在控制 compute 后，模型能力是否提升。

所以上游模型评测可以用参数量、训练 FLOPs、active parameters、训练 compute 等 proxy 来比较。

下游开发者想知道:

- 我部署这个系统每个用户请求要花多少钱。
- 延迟是否可接受。
- token 输入输出是多少。
- 工具调用和外部 API 成本是多少。
- 同样任务能否用 RAG、long context、workflow 或缓存更便宜地完成。

所以对下游开发者来说，真实 dollar cost 不是噪声，而是目标本身。

论文用 NovelQA 做例子。NovelQA 的小说长度从 50,000 到超过 1,000,000 words，每本小说有 5 到 100 个问题。这个 benchmark 很适合评估 long-context model 是否能读长文并回答问题。

但如果一个产品要做“用户连续询问小说内容的问答机器人”，用户通常不会一次性提交所有问题。用户会一个问题一个问题地问。如果每次都把整本小说重新塞进 context，成本会比 benchmark 里的批量问答高很多。

论文发现 long-context 和 RAG 在准确率上大致接近，但 RAG 在真实 sequential QA 场景中便宜超过 20 倍。可是 NovelQA 的 leaderboard 方式会让 RAG 看起来只便宜一半左右，从而严重高估 RAG 成本。

这一节给学习者的关键启发是：

benchmark task shape 不等于 product task shape。

只要输入组织方式不同，成本结论就可能完全变。

5.4 贡献四: agent benchmark 很容易产生 shortcut

普通监督学习 benchmark 已经会过拟合。agent benchmark 更危险，因为 agent 可以利用环境、网页结构、测试格式、工具接口、任务重复模式等 shortcut。

论文提出四个 generality level。

Distribution-specific:

- 只评估同一任务同一分布。
- 适合 hold out in-distribution samples。
- 例如固定类型的小学数学题。

Task-specific:

- 任务固定，但要考虑分布漂移。
- 适合 hold out out-of-distribution samples。
- 例如订机票、下订单、解决 GitHub issue。

Domain-general:

- 目标是在一个 domain 内完成多种任务。
- 适合 hold out tasks。
- 例如 web browsing 或 tool use。

Fully general:

- 目标是跨 domain 通用。
- 适合 hold out domains。
- 例如同一个 agent 同时做 web、robotics、coding、retrieval。

Table 1 的关键结论是: benchmark 的泛化目标越高, holdout 和训练集之间的差异也应该越大。domain-general benchmark 不能只 hold out 同分布样本，因为这无法测试 unseen task 泛化。

论文调查 17 个 agent benchmark，发现多数没有合适的 holdout，其中 7 个没有 holdout，也没有说明未来会加。这意味着很多 leaderboard 分数可能高估了真实泛化。

5.5 贡献五: agent evaluation 缺乏标准化和可复现性

作者在复现 HumanEval 和 WebArena 相关结果时发现很多问题。

五类根因是:

- evaluation script 假设某种 agent 设计，别的 agent 不一定满足。
- 把 LLM benchmark 改成 agent benchmark 时引入不一致。
- agent eval 成本高，导致很难重复多次并报告置信区间。
- 外部环境交互带来 subtle errors，比如 rate limit 和任务顺序依赖。
- 缺乏标准流程导致 agent 开发和评测中出现 bug。

论文提到 SWE-Agent 每个任务可以有几美元级别成本，SWE-bench 又有大量任务，因此做完整多次运行很贵。成本压力会让论文少报告 error bars，也会让 reproduction 更困难。

WebArena 的环境交互问题也很典型。比如 Reddit clone 有 rate limit，如果涉及发帖的任务连续执行，后面的任务更容易失败。这说明 agent benchmark 的样本不一定独立，任务顺序也可能影响结果。

作者还提到 LATS 和 STeP 的评测中有把错误任务标成正确、删除少量任务等问题。这些并不一定是恶意，但足以说明没有标准化框架时，agent leaderboard 很容易被细节污染。

6. Figure 1: HumanEval 上的成本受控比较

Figure 1 是论文最重要的图之一。它把 HumanEval 上的 agent 放到 accuracy-cost 平面里，而不是只按 accuracy 排名。

实验设置可以理解成:

benchmark:

HumanEval, 164 problems

reported metrics:

mean accuracy across 5 runs

mean total cost across 164 problems

compared systems:

zero-shot GPT-3.5 or GPT-4

LDB

LATS

Reflexion

retry baseline

warming baseline

escalation baseline

作者使用 LDB 论文提供的 modified HumanEval 版本，因为这个版本给 164 个任务都提供 examples。原始 HumanEval 只有 161 个任务有 examples。

这件小事也很重要: agent baseline 经常依赖 visible tests 来判断是否 retry 或 debug。如果 benchmark version 不同，结果就不能直接比较。

Figure 1 支持三个结论。

第一, 所谓 SOTA agent architecture 在 HumanEval 上没有明显超过简单 baseline。warming strategy 和表现最好的 agent architecture 之间没有显著 accuracy 差异。

第二, 成本差异巨大。在类似准确率下, 不同方法的成本可以差近两个数量级。Reflexion 和 LDB 比 warming 贵超过 50%, LATS 比 warming 贵超过 50 倍。

第三, escalation baseline 很有启发。它先用便宜模型, 不行再升级到贵模型。这样可以提高准确率, 同时成本低于某些复杂 agent 的一半。

作者真正想打击的是一个常见叙事:

agent 引入 planning, reflection, debugging
所以更像 System 2 reasoning
所以 HumanEval 分数提升证明 System 2 有效

论文的回答是:

不一定。

如果你没有和 retry, warming, escalation 比,
你无法知道 gain 是来自 System 2 架构,
还是来自更多采样, 更贵模型, 更多 token。

这不是说 System 2 方法在更难任务上无用。作者明确承认 SWE-bench 等更复杂任务可能更需要真实 planning 和 debugging。但 HumanEval 上的证据不足以支持很多强叙事。

7. Pareto frontier 怎么读

这篇论文把 agent 评测从单指标排名改成多指标设计。

定义一个 agent run:

```
run = {  
    accuracy: task success rate  
    variable_cost: per-task inference cost  
    fixed_cost: one-time optimization cost  
    latency: wall-clock delay  
}
```

如果 agent A 满足:

```
A.accuracy >= B.accuracy  
A.cost <= B.cost  
and at least one inequality is strict
```

那么 A dominates B。B 被支配, 就不应该是首选设计。

Pareto frontier 是所有未被支配的点。

```
accuracy  
^  
|  
|           expensive high-accuracy agent  
|           o  
|  
|   o Pareto frontier  
|   o  
|   o
```


+-----> cost
cheap expensive

dominated point:

if another point is both cheaper and at least as accurate

论文还提到一个细节: 可以把 frontier 约束成 convex, 因为可以按概率混合两个 agent。比如一半请求用 agent A, 一半请求用 agent B, 就得到两点之间的平均 cost 和 average accuracy。

新手容易误读 Pareto:

- Pareto frontier 不是一个唯一最优点。
- 它是一组候选点。
- 真正选择哪个点, 要看你的业务成本权重、延迟要求、风险要求。
- 被支配的点通常不值得作为默认选择。

8. fixed cost 和 variable cost

第 3 节强调 agent 成本不止 inference cost 一种。

fixed cost:

- 调 prompt。
- 搜索 hyperparameter。
- 选择 few-shot examples。
- 做开发集优化。
- 建索引或写规则。
- 人工调试和 benchmark submission。

variable cost:

- 每个任务输入 token。
- 每个任务输出 token。
- 每次工具调用。
- 每次模型 retry。
- 每次环境交互。

总成本:

$\text{total_cost}(n) = \text{fixed_cost} + n * \text{variable_cost}$

$\text{amortized_cost}(n) = \text{fixed_cost} / n + \text{variable_cost}$

当使用次数很少时, fixed cost 很重要。一个优化器花了很多钱找 prompt, 如果只运行 100 个任务, 摊销成本可能很高。

当使用次数很大时, variable cost 主导。产品部署后每天处理大量请求, 就算 fixed cost 较高, 只要 variable cost 足够低, 也可能值得。

这就是论文在 HotPotQA 上讨论“joint optimization becomes cheaper after enough tasks”的原因。作者报告 joint optimization 在某个使用量之后会比默认 DSPy 更便宜, 这个点来自 fixed cost 被摊薄。

9. Figure 2: HotPotQA 和 DSPy 联合优化

HotPotQA 是多跳问答任务，通常需要检索和综合多个证据。论文用 ColBERTv2 over Wikipedia 作为 retriever，并修改 DSPy 优化过程。

实验可以按这个流程理解：

```
data:
    100 training examples for optimization
    200 evaluation examples

agent modules:
    question handling
    retrieval
    answer generation

search space:
    temperature per module
    number of few-shot examples
    which examples to include
    whether to add formatting instructions

optimizer:
    Optuna search

objective:
    find Pareto-optimal configs on accuracy and cost
```

比较的 agent design 包括：

- Uncompiled.
- Formatting instructions only.
- Few-shot.
- Random Search.
- Joint optimization.

论文使用 Llama-3-70B 和 GPT-3.5 两类模型。核心结果不是“某个方法准确率最高”，而是“在准确率差不多时，joint optimization 更便宜”。

这节对工程最有价值的地方是：prompt optimizer 不应该只优化 accuracy。否则它可能不断增加 few-shot examples，导致每次调用的 input tokens 变大。这个配置在 dev set 上看起来更好，但产品部署成本会变差。

一个更好的 optimizer 应该像下面这样想：

```
for each candidate prompt/config:
    measure accuracy
    measure tokens_in and tokens_out
    measure tool calls
    compute variable_cost
    keep Pareto candidates
```

10. NovelQA: benchmark task shape 会误导成本

NovelQA 的例子非常适合训练你识别 benchmark 和产品之间的差异。

benchmark 形状:

```
input:
  entire novel
  all questions for that novel

output:
  all answers

cost:
  long context paid once per novel
```

真实产品形状:

```
input per user turn:
  entire novel again if using naive long context
  one question

output per user turn:
  one answer

cost:
  long context paid once per question
```

RAG 产品形状:

```
offline:
  split novel
  embed chunks
  build index

input per user turn:
  retrieved chunks
  one question

output per user turn:
  one answer
```

所以同一个 benchmark，如果把多个问题一次性打包，会系统性低估 long-context sequential QA 的成本，也会让 RAG 看起来没那么有优势。

论文的结论是: NovelQA 对 model evaluation 有意义，但不能直接替代 downstream evaluation。下游开发者应该构造和用户使用方式一致的 benchmark variant。

这个结论对你学习 agent 很重要。以后看到任何 benchmark，都要问:

- task item 是一次性批处理，还是多轮交互。
- 用户是否会顺序提出问题。
- 每轮是否重复输入大 context。
- 是否有缓存、RAG、memory、indexing。
- benchmark 是否测了产品真实成本。

11. Table 1: holdout 必须匹配泛化目标

这篇论文最实用的表是 Table 1。它把 agent benchmark 的泛化目标和 holdout 类型对齐。可以记成下面的规则。

If benchmark claims distribution-specific:
hold out in-distribution samples.

If benchmark claims task-specific:
hold out out-of-distribution samples.

If benchmark claims domain-general:
hold out tasks.

If benchmark claims fully general:
hold out domains.

为什么传统 ML 常常 hold out samples 就够了？因为传统模型通常只做单一任务，例如图像分类或垃圾邮件检测。测试集样本独立同分布，基本能估计同分布泛化。

为什么 agent 不够？因为 agent 通常宣称能处理未知自然语言任务、未知工具组合、未知网页结构。此时 hold out 同分布样本只能测试“同类样本复制能力”，不能测试“新任务适应能力”。

举例：

- WebArena 如果被当作 web domain-general benchmark，就应该 hold out unseen web tasks 或 unseen websites。
- tau-bench 如果被当作 tool-agent-user interaction 的 domain-general benchmark，就应该逐步加入 unseen domains 和 unseen tasks。
- GAIA 如果被当作 fully general assistant benchmark，只 hold out 同类问题可能不足以支持 fully general 叙事。

你以后评价一个 agent benchmark，可以先写下：

claimed generality:
distribution-specific / task-specific / domain-general / fully-general

actual holdout:
samples / OOD samples / tasks / domains / none

interpretation:
aligned / underpowered / shortcut-prone

12. WebArena 和 STeP case study

WebArena 是 web agent benchmark。它包含六类网站 clone：

- GitLab。
- Reddit。
- Wikipedia。
- OpenStreetMaps。
- e-commerce platform。
- content management system。

它还有两个工具:

- calculator。
- scratchpad。

任务数量是 812。任务例子包括在网页中查找信息、发帖、导航、操作表单等。

WebArena 的卖点是 realism。但论文指出, 如果没有合适 holdout, 这种 realism 也可能被 shortcut 利用。

STeP 是 WebArena leaderboard 上表现很强的 agent。论文说它达到 35.8% accuracy, 是 WebArena 原论文 top baseline 的两倍多, 也比下一个 agent 高 10 个百分点以上。

问题在于: STeP hardcodes policies for specific WebArena tasks。

例如某些 Reddit profile task 可以通过在当前 base URL 后加 /user/user_name 解决。这在 benchmark clone 上有效, 但如果真实网站改了 URL 结构, 这个 policy 就会失效。

作者并没有攻击 STeP 的工程目标。固定任务上写 composable policies 是合理的。但如果 leaderboard 被解读成“这个 agent 在真实 web tasks 上泛化更强”, 那就误导了下游开发者。

关键教训:

Hardcoded policy can be useful engineering.

But it is not evidence of domain-general web-agent ability.

13. human-in-the-loop evaluation

论文还指出 agent benchmark 常常走两个极端:

- 只测 chatbot 正确率。
- 或测完全 autonomous agent。

真实产品中, 人通常在 loop 里:

- 人给目标。
- agent 提出计划。
- 人修正计划。
- agent 执行动作。
- 高风险动作需要人确认。
- 失败后人给反馈。

human-in-the-loop 既可能让 agent 更安全, 也可能显著提高成功率。论文提到在某些困难 programming 问题上, 简单人类反馈能把 GPT-4 从完全失败提高到接近完美的表现。

这会带来双向偏差。

如果 benchmark 完全没有 human-in-the-loop:

- 它可能高估全自动 agent, 因为没有测真实监督成本。
- 它也可能低估辅助型 agent, 因为人类反馈能大幅提高成功率。

所以一个下游产品评测应该明确:

autonomy level:

```
no human intervention
human can approve actions
human can correct plan
human can provide observations
```

human can recover failed states

measured cost:

model cost
tool cost
latency
human time

14. 可复现性为什么在 agent 中更难

普通 LLM benchmark 常常是:

input string -> model -> output string -> scorer

agent benchmark 更像:

task instruction
-> agent state
-> model call
-> tool call
-> environment mutation
-> observation
-> another model call
-> action sequence
-> final state
-> scorer

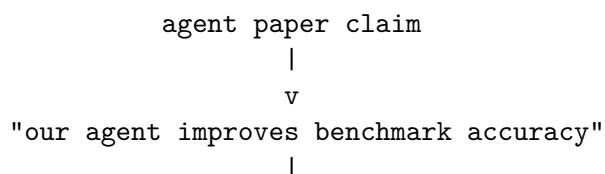
这多了很多不稳定因素:

- 工具 API 版本。
- 网页结构。
- 环境初始状态。
- 任务执行顺序。
- rate limit。
- 模型随机性。
- hidden prompt。
- visible tests 是否提供。
- scorer 如何判定完成。
- 出错任务是否被删除。

论文要求的 reproducibility 是: 论文附带的 code 和 data 应该足以复现报告结果。这个标准听起来基础, 但在 agent 领域很难, 因为很多结果依赖外部环境和隐藏交互细节。

作者认为 agent 领域需要专门标准, 而不仅是套用 HELM 或 LM Evaluation Harness。原因是这些工具主要面向模型输入输出, 不覆盖长链 action、环境 mutation、tool state 和 human-in-the-loop。

15. 全文机制图



```

      v
+-----+
| Questions this paper asks |
+-----+
| Did it beat retry/warming/escalate? |
| Was cost measured? |
| Is it on Pareto frontier? |
| Is benchmark task shape realistic? |
| Is holdout aligned with generality? |
| Can others reproduce the score? |
+-----+

```

```

      |
      v
credible agent progress

```

一个更工程化的评测流:

```

task set
|
v
run baseline agents
  zero-shot
  retry
  warming
  escalation
|
v
run proposed agent
|
v
collect logs
  accuracy
  token counts
  tool calls
  latency
  failures
  human interventions
|
v
plot Pareto frontier
|
v
interpret only non-dominated configs

```

16. 数据形状和评测对象

把论文转成数据结构，可以这样想。

HumanEval item:

```

problem = {
  prompt: str,

```

```

    visible_tests: list[str],
    hidden_tests: list[str]
}

```

```

agent_output = {
    candidate_code: str,
    attempts: list[str],
    tool_calls: list[str],
    tokens_in: int,
    tokens_out: int,
    cost_usd: float
}

```

```

score = {
    pass_hidden_tests: bool,
    total_cost_usd: float
}

```

HotPotQA item:

```

example = {
    question: str,
    supporting_facts: list[str],
    answer: str
}

```

```

agent_config = {
    prompt_template: str,
    few_shot_examples: list[example],
    module_temperatures: list[float],
    formatting_instructions: bool
}

```

```

measurement = {
    exact_match_or_f1: float,
    tokens_in: int,
    tokens_out: int,
    cost_usd: float
}

```

WebArena item:

```

task = {
    instruction: str,
    website: str,
    initial_state: environment_state,
    allowed_tools: list[str],
    success_condition: str
}

```

```

agent_trace = {
    observations: list[str],

```



```

    actions: list[str],
    final_state: environment_state,
    cost_usd: float,
    errors: list[str]
}

```

NovelQA downstream item:

```

user_session = {
    long_document: str,
    sequential_questions: list[str]
}

```

```

long_context_cost:
    cost of sending full document each question

```

```

rag_cost:
    offline indexing cost
    plus retrieved chunks per question

```

这些数据形状帮助你看清楚: agent evaluation 评的不是一个单次 string prediction, 而是一条带成本、状态和环境副作用的 trajectory。

17. 数学化理解

17.1 成本受控效用

下游产品可以用一个简单 utility 表达:

```

utility(agent) =
    accuracy
    - c_cost * amortized_cost
    - c_latency * latency
    - c_risk * risk

```

其中:

- accuracy 是任务成功率。
- amortized_cost 是 fixed cost 摊销后加 variable cost。
- latency 是平均延迟。
- risk 可以是安全违规、不可复现、shortcut 风险等。
- c_cost、c_latency、c_risk 是业务权重。

这不是论文给出的唯一公式, 而是把论文思想写成工程可用形式。它告诉你: agent 选择是多目标优化, 不是单纯 accuracy 排名。

17.2 Pareto 支配

A dominates B if:

```

accuracy_A >= accuracy_B
cost_A <= cost_B
and one side is strictly better

```

如果 B 被支配，说明至少存在一个 agent A 更便宜且不差，或者更准且不贵。此时继续宣传 B 的复杂架构没有太大意义。

17.3 使用量摊销

```
total_cost(n) = fixed_cost + n * variable_cost
amortized_cost(n) = fixed_cost / n + variable_cost
```

当 n 很小，fixed cost 决定是否值得优化。

当 n 很大，variable cost 决定产品能否长期承受。

17.4 downstream cost 不是 proxy

模型评测中，参数量或 compute 可以是 proxy，因为研究者想控制训练资源。

下游评测中，真实 dollar cost 是被优化目标。即使模型价格会变，也应该报告：

- dollar cost at measurement time.
- input token count.
- output token count.
- tool calls.

这样未来价格变化后，别人可以重新计算成本。

18. 本地代码例子

本仓库新增了一个小模块：

```
learning/agent-graduation/src/eval/agent_eval_matter.py
```

它把论文主张转成几个可运行函数：

- AgentRun: 记录 accuracy、variable cost、fixed cost、latency。
- pareto_frontier: 找出未被支配的 agent configs。
- cost_controlled_score: 用 utility 选择下游配置。
- required_holdout: 根据 generality level 返回正确 holdout。
- benchmark_shortcut_risk: 给 benchmark 解释风险打分。
- novelqa_cost_scenarios: 比较 batched benchmark 和 sequential downstream cost。

最小代码片段：

```
from eval.agent_eval_matter import AgentRun, pareto_frontier

runs = [
    AgentRun("complex-agent", accuracy=0.91, variable_cost_usd=0.90),
    AgentRun("warming", accuracy=0.91, variable_cost_usd=0.30),
    AgentRun("escalation", accuracy=0.93, variable_cost_usd=0.25),
]

frontier = pareto_frontier(runs)
print([run.name for run in frontier])
```

预期你会看到 complex-agent 被排除，因为它和 warming 准确率相同但更贵，而且 escalation 还更准更便宜。

NovelQA 成本错觉的最小片段:

```
from eval.agent_eval_matter import novelqa_cost_scenarios
```

```
costs = novelqa_cost_scenarios(  
    novel_tokens=200_000,  
    question_tokens=80,  
    answer_tokens=120,  
    n_questions=20,  
    retrieved_tokens_per_question=2_000,  
    price_in_per_mtok=1.0,  
    price_out_per_mtok=3.0,  
)
```

```
for name, cost in costs.items():  
    print(name, cost)
```

你应该观察到:

- batched benchmark cost 很低, 因为小说只输入一次。
- long-context sequential cost 很高, 因为每个问题都重新输入小说。
- RAG sequential cost 明显更低, 因为每次只输入 retrieved chunks。

19. 和本地 agent-graduation 代码的关系

这个专题本来已经有三个 capstone 层次。

Deep Research Agent:

learning/agent-graduation/src/dra/

它包含 planner、retriever、writer、verifier、tool set 和 orchestrator, 模拟一个 research agent 的完整流程。

Tau-bench style eval:

learning/agent-graduation/src/eval/tau_bench_mock.py

learning/agent-graduation/src/eval/dra_eval.py

它用五个 mock tasks 和五个维度评估 agent:

- goal completion。
- tool use。
- safety。
- efficiency。
- cost。

Portfolio:

learning/agent-graduation/src/portfolio_v2.py

它把整个学习仓库的主题串成毕业 portfolio。

新增模块的作用是把论文的评价哲学接到这些 capstone 上:

DRA is not enough.

Tau-style task score is not enough.

We also need:

- cost-controlled baselines
- Pareto frontier
- holdout alignment
- reproducibility checklist
- downstream task-shape cost

20. 30 到 60 分钟本地实验

实验目标: 复现论文最核心的思维方式, 而不是复现全部 HumanEval。

步骤 1: 跑环境检查。

```
python learning\agent-graduation\environment\verify_env.py
```

步骤 2: 跑专题测试。

```
python learning\agent-graduation\src\tests\test_graduation.py
```

步骤 3: 单独跑新增模块。

```
python learning\agent-graduation\src\eval\agent_eval_matter.py
```

步骤 4: 修改 agent_eval_matter.py 里的 demo runs。

建议你做三个改动:

- 把 complex-agent 的 accuracy 提高到 0.96。
- 把 optimized-agent 的 fixed cost 从 50 改成 500。
- 把 n_runs 从 10 改到 10,000。

观察:

- 哪些点进入 Pareto frontier。
- 使用量变大后 fixed cost 如何被摊薄。
- 高 accuracy 是否值得高 variable cost。
- best_by_utility 是否和 Pareto frontier 一致。

步骤 5: 写 100 字实验结论。

模板:

我发现某 agent 在短期使用下不划算, 因为 fixed cost 尚未摊销。
但在长期使用下, 它进入 Pareto frontier。
这说明 agent 论文不能只报告一个 accuracy。
必须报告成本、使用量假设和 baseline。

21. 证据链逐段复盘

论文证据链不是一个单表格, 而是五段互相支撑。

第一段证据: HumanEval。

- 问题: 复杂 coding agent 是否真比简单 baseline 好。
- 设计: 用 accuracy-cost Pareto 比较。
- 关键 baseline: retry、warming、escalation。
- 发现: 复杂 agent 没有明显击败强简单 baseline, 成本差异巨大。

- 结论: 不控制成本和 baseline, 就无法归因到架构。

第二段证据: HotPotQA 和 DSPy。

- 问题: cost-aware optimization 是否能产生更好设计。
- 设计: 搜索 prompt、few-shot、temperature、formatting。
- 指标: accuracy 和 variable cost。
- 发现: joint optimization 保持 accuracy, 同时降低 cost。
- 结论: 成本可以进入优化目标, 而不是事后报告。

第三段证据: NovelQA。

- 问题: model benchmark 是否能直接指导 downstream product。
- 设计: 比较 benchmark batched QA 和真实 sequential QA。
- 发现: RAG 在真实场景中便宜超过 20 倍, 但 benchmark 会弱化这种优势。
- 结论: task shape 必须匹配产品 workflow。

第四段证据: benchmark holdout survey 和 WebArena case。

- 问题: agent benchmark 是否防 shortcut。
- 设计: 四级泛化目标和 holdout 对齐。
- 发现: 多数 benchmark 缺少合适 holdout。
- case: STeP 在 WebArena 上使用 hardcoded policies。
- 结论: leaderboard 高分不等于 domain-general 能力。

第五段证据: reproducibility audit。

- 问题: agent evaluation 是否可复现。
- 设计: 复现 HumanEval 和 WebArena 相关结果。
- 发现: 脚本假设、benchmark 改造、成本、环境交互、评测 bug 都会影响结果。
- 结论: agent 需要专门的标准化评测框架。

22. 论文没有证明什么

这篇论文也有边界。

它没有证明复杂 agent 架构都没用。它只证明在他们分析的若干场景里, 现有证据不足以支持很多强说法。

它没有给出所有 agent benchmark 的最终解决方案。它提出原则: cost control、strong baselines、proper holdouts、standardization。

它没有穷尽所有任务。HumanEval 是代码生成 benchmark, HotPotQA 是问答 benchmark, NovelQA 是长文本问答 benchmark, WebArena 是网页任务 benchmark。更难的 SWE-bench、真实企业 workflow、长期 memory task 可能有不同结果。

它没有消除成本模型变化。模型价格会变, 硬件会变, API 会变。所以作者建议报告 token count 和 dollar cost, 让后人可以重新计算。

它没有深入分析所有成本。论文主要关注推理 dollar cost, 也提到其他成本没有完全覆盖, 例如环境成本、人力成本、维护成本和部署复杂度。

23. 对现在学习 LLM agent 的意义

这篇论文最像一套 agent 学习免疫系统。

当你用 AI agent 加速学习时, 它能帮你避免三种坑。

第一种坑: 被 agent 生成的漂亮复杂流程迷惑。你要让 agent 给出 baseline, 并解释为什么复杂模块有必要。

第二种坑: 把 demo 成功当作能力。你要让 agent 写出评测 task set、失败案例、成本统计和可复现脚本。

第三种坑: 只追新 benchmark。你要让 agent 判断 benchmark 的 claimed generality 和 actual holdout 是否一致。

对本仓库的学习节奏来说, 这篇应该放在 agent 模块后半段。你先学 ReAct、Toolformer、MemGPT、agent framework survey, 再读这篇, 等于给前面所有 agent 能力加上评价准则。

读完它以后, 你对任何 agent 项目都应该能问:

- 这个 agent 在什么任务上 matters。
- 它对谁 matters, 研究者还是下游开发者。
- 它和什么 baseline 比。
- 它的每次运行成本是多少。
- 它有没有被更便宜同准确率的方法支配。
- 它的 benchmark 有没有 shortcut。
- 它的结果别人能不能复现。

24. 用 AI agent 正确学习这篇

你可以把下面这段发给学习 agent, 但要先自己读一遍本 guide。

我正在学习 AI Agents That Matter。

请不要泛泛总结。

请按下面顺序考我:

1. 为什么 accuracy-only agent benchmark 会误导?
2. retry, warming, escalation 三个 baseline 分别解决什么问题?
3. Pareto frontier 如何改变 agent 选型?
4. fixed cost 和 variable cost 为什么要分开?
5. NovelQA 为什么适合 model eval 但会误导 downstream eval?
6. Table 1 的四级 generality 和 holdout 如何对应?
7. WebArena/STeP case 说明了什么 shortcut 风险?
8. agent evaluation 为什么比普通 LLM evaluation 更难复现?

每次只问一个问题。

我回答后, 请指出我是否把论文证据和本地代码对应起来。

最后让我用 200 字闭卷复述这篇论文。

更好的 agent 学习法是让 agent 当严厉助教, 而不是当摘要机器。

你可以要求它:

- 不接受“agent 要看成本”这种空话。
- 必须让你说出 HumanEval、HotPotQA、NovelQA、WebArena 各自证明什么。
- 必须让你画出 Pareto frontier。
- 必须让你解释一个被支配点为什么不该被宣传。
- 必须让你打开 agent_eval_matter.py 改一个参数。

25. 闭卷掌握检查

如果你真的掌握这篇论文，应该能不看 guide 回答这些问题。

1. 这篇论文为什么不满足于比较 benchmark accuracy?
2. retry、warming、escalation 为什么是强 baseline? 它们分别利用了什么机制?
3. 为什么一个复杂 agent 和 warming 准确率相同但成本更高时，不能算好设计?
4. Pareto frontier 是什么? 被支配点意味着什么?
5. fixed cost 和 variable cost 的区别是什么? 为什么使用量会改变最优选择?
6. HotPotQA 的 DSPy joint optimization 搜索了哪些设计变量?
7. NovelQA 的 benchmark task shape 和真实 sequential QA 有什么区别?
8. 为什么参数量或 active parameters 对 downstream cost 可能是坏 proxy?
9. Table 1 中 domain-general benchmark 应该 hold out 什么?
10. WebArena/STeP 例子为什么不是简单地说 STeP 做错了?
11. human-in-the-loop evaluation 为什么既可能提高成功率，也会增加评测复杂度?
12. agent evaluation 的五类不可复现根因是什么?
13. 如果你要评估本仓库 DRA agent，你会加入哪三个 cost-controlled baseline?
14. 如何把本论文思想转成 agent_eval_matter.py 里的一个 toy experiment?
15. 这篇论文最重要的现代意义是什么?

26. 最小复述模板

闭卷时可以用这个结构讲:

AI Agents That Matter 认为 agent 领域不能只用 accuracy 排名。因为 agent 可以通过更多调用、更贵模型、retry、benchmark shortcut 来提高分数。作者在 HumanEval 上显示，retry、warming、escalation 这些简单 baseline 就能和复杂 agent 竞争，而且成本更低。因此 agent 设计应放到 accuracy-cost Pareto frontier 上比较。作者又在 HotPotQA/DSPy 上说明成本可以进入优化目标，在 NovelQA 上说明 model benchmark 会误导 downstream cost，在 WebArena/STeP 上说明缺少正确 holdout 会产生 shortcut。最后，论文指出 agent evaluation 还存在标准化和复现问题。它的意义是把 agent 进步从 demo 和 leaderboard 拉回成本、泛化、复现和真实产品价值。

如果你能用自己的话讲出这段，并能打开本地代码跑 agent_eval_matter.py，这篇就不是只进了收藏夹，而是真的进了脑子。